HPC setup checklist (UVA / remote + local workflow)

This document pulls together how **Git**, **conda**, **large data**, **Cursor/SpecStory**, and **storage** (Dropbox, OneDrive, HPC) fit together when you work on the tenure project **on a cluster**, **over VPN**, or **sometimes on your Mac**. It is meant to be read once for orientation and used as a step list when you stand up a new machine.

Pipeline semantics (cells, stages, `tenure_pipeline/` outputs): see `TENURE_PIPELINE_OVERVIEW.md` in this same folder (`tenure/documents/`).

---

1. What problem we are solving

You want:

- **The same code** on your laptop and on the HPC, without emailing zip files.
- **Large scraped and derived files** (JSONL, DBLP-related outputs, snapshots, etc.) **not** bloating GitHub — they stay on disk you control and get **copied** or **regenerated** where you compute.
- **A reproducible Python environment** (`tenure_net` or equivalent) on the HPC, not a mystery install.
- **Optional:** SpecStory (or similar) so conversation exports can live **in the repo path** if you commit them.

Git solves the **small, versioned** part. **rsync** (or manual copy) solves **big data**. **Conda export** solves **environment parity**.

---

2. What Git does *and does not* do

Git *does*

- Stores **tracked** source code, notebooks, docs, and small config **templates** (e.g. `.env.example`).
- Lets you `git clone` on the HPC and get a **synchronized copy of the repository** from GitHub (whatever you last **pushed**).
- Lets you work **locally on your Mac** sometimes and **on the HPC** other times: you `pull` before you start on a machine, work, `commit`, `push`, then on the other machine `pull` again.

Git *does not*

- **Create your conda environment.** You build the env on each machine (or from an exported file — see below).
- **Include** `.env` (secrets). That file is **gitignored**; you create it per machine from `.env.example`.
- **Include large pipeline outputs** that we deliberately keep out of the repo. The root `.gitignore` excludes things like multi-GB DBLP dumps, large JSONL panels, `**/faculty_snapshots/`, many paths under `tenure/tenure_pipeline/`, etc. `git clone` **alone will not give you those files.**

One mental model

Need	Tool
Code + small tracked files	`git pull` / `git push`
Secrets	`.env` on each system (never commit)
Huge data	`rsync`, tarball, or re-run the pipeline on the HPC
Python packages	`conda` / `pip` using an **exported** env or requirements

Shell scripts (`.sh`) and Playwright PDF helpers

- **Nothing in `.gitignore` excludes `*.sh`.** Your repo includes many scripts under `scripts/` (e.g. `generate_pdf_playwright.sh`, `convert_*_md_to_pdf.sh`). If those paths are **tracked** (`git add` / `commit` / `push`), they are **part of the clone** on any machine — Mac, HPC, or elsewhere.
- **"Works"** still depends on the **environment**, not only Git:
  - The Playwright-based flows need `pip install playwright` (or equivalent in conda) and `playwright install chromium` (browser binaries). Put those in your `environment-tenure_net.yml` if you want them on the HPC.
  - **Headless Chromium** on a cluster is sometimes **restricted** or awkward (no display, no outbound network, compute nodes vs login node). Many people run **PDF / Markdown → PDF** pipelines on **your Mac** or an **interactive session** and use the HPC for **heavy compute** (notebooks, parsing, large jobs). If Playwright fails on the batch nodes, try a **login** or **interactive** node, or keep doc conversion local.
- **Naming:** If you meant **Markdown → PDF** (not "md to py"), that matches `generate_pdf_playwright.sh` and the `convert_*_md_to_pdf.sh` wrappers. If you have a separate **md → py** tool, that is whatever script you run — same rule: if it is committed, it lives in the clone.

---

3. Suggested order: first-time HPC workspace

Use this when you open a **new** project directory on the cluster (after you know where you are allowed to store data: home vs project vs scratch).

Which machine runs which step?

Step	Typical machine	What you are doing
A	**Any** (browser) + **first SSH to HPC**	Read UVA / Research Computing docs (quotas, scratch, VPN) **on the web**; then **on the cluster**, confirm paths exist (`ls`, `df -h`) and where you may put the repo and big data.
B	**HPC** (SSH session)	`git clone` into the path you chose.
C (export)	**Mac** (once)	`conda env export` → commit `environment-tenure_net.yml`.
C (create env)	**HPC**	`conda env create` / `conda activate`.
D	**HPC**	Create `.env` from `.env.example`.
E	**Mac → HPC** (or regenerate on HPC)	`rsync` or pipeline runs for large data.
F	**Either**	SpecStory files in repo vs cloud; same clone on HPC after push.
G	**HPC**	Smoke test imports / small script.

Your Mac is where you edit and push **before** the HPC sees new commits (unless you commit on the HPC). **VPN** matters when you are **off campus** and need to reach UVA networks to SSH.

Step A — Know policy, quotas, and paths (not "install Git")

Step A is not about installing software on your laptop. It is about **understanding the cluster** before you put terabytes in the wrong place.

- **Where (read documentation):** Use **any machine with a browser** — log into **UVA Research Computing** (or your HPC's) documentation and read:
  - **Home vs project vs scratch** storage: what is **backed up**, what **expires**, and **quotas**.
  - Whether you need **VPN** from off campus to reach SSH.
  - **Login node vs compute node** vs **interactive** jobs (you usually **clone** and **edit small files** on a login node; heavy jobs go to the scheduler — but policies vary).
- **Where (verify on the cluster):** After you **SSH into the HPC** (`ssh …@…`), confirm the paths you plan to use exist and have space, e.g. `pwd`, `ls`, `df -h` on your home or project directory. This step is **on the Linux login**, not on your Mac's Finder.
- **Why:** Avoid putting the only copy of huge data in **scratch** if it is purged after N days, or filling **home** past quota.

Once Step A is clear, you know **which directory on the HPC** is correct for `git clone` in Step B.

Step B — `git clone`

Clone your GitHub (or GitLab) repo into that location. This gives you **code and tracked files** only — not gitignored data.

Where this runs: You usually **type these commands in a terminal that is already connected to the HPC** — e.g. `ssh you@cluster` from **your Mac's** Terminal, or the **integrated terminal in Cursor Remote SSH**. The clone is created **on the cluster's filesystem** (Linux paths), not in your Mac's `~/Documents` unless you intentionally run `git clone` **locally** instead (that would be a different workflow).

```
git clone <your-repo-url>
cd <repo-directory>
```

Step C — Recreate the Python environment (`tenure_net`)

Git does not install packages. On your **Mac**, with `tenure_net` activated, export the environment so the HPC can recreate it:

```
conda activate tenure_net
conda env export --no-builds > environment-tenure_net.yml
```

- `--no-builds` keeps the file more portable across Linux vs macOS (fewer hard pins that break on the other OS).
- Save `environment-tenure_net.yml` **inside the repo** (e.g. `tenure/documents/` or repo root), then **stage, commit, push** so it is available after every `git clone`.

On the **HPC**, after cloning:

```
conda env create -f environment-tenure_net.yml -n tenure_net
conda activate tenure_net
```

If `conda env create` complains about a few packages on Linux, fix the yml once (or install those packages manually) and update the committed file.

Fallback: the repo's root `requirements.txt` is **minimal** (e.g. `python-dotenv` only). It is **not** a full substitute for a conda export of `tenure_net`.

Step D — `.env` on the HPC

Copy from `.env.example` at the repo root, fill in keys (OpenAlex, email, etc.). Never commit `.env`.

Step E — Large / scraped data

Anything listed in `.gitignore` under tenure pipeline / datasets **must** be supplied separately:

- **Copy from your Mac** with **rsync** (see section 5), **or**
- **Regenerate** on the HPC by running the appropriate notebook/cells (slower but reproducible), **or**
- **Stage an intermediate:** copy only the artifacts needed for the next step.

Match the **same relative paths** your code expects (e.g. under `tenure/tenure_pipeline/`).

Step F — SpecStory / conversation history

- If SpecStory (or similar) saves **markdown or project files inside the repo** and you **commit** them, then `git clone` on the HPC puts those files in the **project path**, and the extension can recognize them.
- **Cloud sync** (SpecStory account) is separate: it does not replace Git, but it can carry history when you use the same login. **Git** is what guarantees files sit in the cloned tree.

Step G — Smoke test

- `python -c "import pandas, dotenv"` (or whatever matters).
- Run a **small** notebook cell or script that does not need the full multi-GB dump.

---

4. Day-to-day: local Mac vs HPC

You can **split work** between machines; Git is designed for that.

Habit:

1. Before editing: `git pull` (get latest from GitHub).
2. After a coherent chunk of work: `git add`, `git commit`, `git push`.
3. On the other machine: `git pull` again.

Caveats:

- **Notebook merge conflicts** can be annoying. Commit often; consider clearing outputs before commit if you want cleaner diffs (optional team preference).
- **Large data** does not move with `git pull` — use **rsync** when you refresh big folders.

---

5. rsync — what it is and why we use it

`rsync` is a command-line utility (available on macOS and Linux) that **copies directory trees** and is excellent for **large** and **repeated** transfers.

Why not only Git or only drag-and-drop?

- Git is wrong for **multi-GB** binaries and huge JSONL — it balloons the repo and GitHub limits apply.
- `rsync` can copy **only what changed** on the second run (**incremental**), which saves hours when you tweak large datasets.

Typical use: Mac → HPC over SSH

You already use SSH to log in; `rsync` can use the **same** host and paths.

Example shape (replace user, host, and paths with yours):

```
rsync -avz --progress \
  "/path/on/Mac/to/tenure/tenure_pipeline/SOME_LARGE_FOLDER/" \
  yournetid@rivanna.hpc.virginia.edu:/path/on/HPC/to/repo/tenure/tenure_pipeline/SOME_LARGE_FOLDER/
```

- `-a` — archive (preserves structure, times, etc.).
- `-v` — verbose.
- `-z` — compress during transfer (helps on slow links).
- `--progress` — optional; shows progress.

Pull from HPC to Mac by swapping source and destination.

When to use rsync

- Refreshing **gitignored** artifacts between laptop and cluster.
- Initial **seed** of a big folder before running jobs on the HPC.

When something else is fine

- Tiny files: `scp` or a GUI is enough.
- **Secrets:** keep API keys in `.env`, not inside huge rsync'd folders, unless you fully trust that directory.

---

6. Dropbox vs OneDrive (UVA) vs HPC vs GitHub — roles

None of these need to "win" exclusively; they have **different jobs**.

Dropbox (personal)

- What you have used for years with your spouse: **sync between your devices**, familiar layout.
- Your dissertation folder may **live** there on the Mac; that is fine for **local** work.
- The HPC **does not** magically see `~/Dropbox` unless you **copy** data there or use a site-specific mount (often **not** available). Treat Dropbox as **your laptop-side** or **manual copy source**, not as the cluster's native filesystem.

OneDrive (University of Virginia)

- **Institutional** storage: large quota, policy-backed, good for **official** research materials, **backups** of important outputs, and anything you want tied to the **university** context.
- Strong candidate for **archival copies** of exports, thesis-related files, and **non-secret** large backups — especially if you want everything under **UVA** stewardship.

HPC disk

- **Compute-local**: fast storage next to jobs, **scratch** for big temporary runs (check your center's retention rules).
- Best for **active** processing: clone here, rsync big inputs here, run jobs, then **copy results** somewhere durable (OneDrive / external) if needed.

GitHub (remote Git)

- **Versioned code** and small tracked assets; collaboration; "what we actually committed."
- **Not** the primary home for multi-GB data (by design, via `.gitignore`).

A sensible split for this project

- **GitHub:** code + `environment-tenure_net.yml` (or similar) + optional SpecStory-exported files if you want them versioned.
- **HPC:** working clone + large data for runs.
- **OneDrive:** institutional backup / advisor-facing exports as appropriate.
- **Dropbox:** optional second copy on your side for **personal** continuity; not required for the cluster to function.

7. Capturing the conda export in Git (your plan)

When you run:

```
conda activate tenure_net
conda env export --no-builds > environment-tenure_net.yml
```

place the file **inside the repo**, then:

```
git add environment-tenure_net.yml   # or the path you chose
git commit -m "Add conda environment export for tenure_net (HPC/local parity)"
git push origin main                   # or your branch name
```

After that, every `git clone` brings the file; the HPC can `conda env create -f ...` as in section 3.

Tip: If the first export includes Mac-only packages that fail on Linux, trim or adjust the yml once and re-commit.

---

8. Quick reference: files Git will *not* give you on clone

Consult the repo root `.gitignore` for the authoritative list. It includes patterns such as:

- `**/faculty_snapshots/` (anywhere in the repo)
- Large DBLP XML paths under `python_packages/dblp-parser/` and `datasets/tenure/`
- Specific large JSONL paths listed under `tenure/tenure_pipeline/` (panel, snapshots index, parsed snapshots, `dblp_parsed/dblp_*.jsonl`)
- `.env`
- Various other large or machine-local trees

Note: `.gitignore` names `tenure/tenure_pipeline/` for large JSONL and related lines. `540_tenure_pipeline` **Cell 0** sets `TENURE_PIPELINE_DIR = WORKSPACE_ROOT / "tenure" / "tenure_pipeline"` — that is the **canonical** on-disk tree for pipeline data and the `tenure_pipeline` **Python package**. An older **duplicate** `tenure_pipeline/` folder at the **repository root** may still exist on disk; Cell 0's `sys.path` order prefers `tenure/tenure_pipeline/` for imports. **Rsync** `tenure/tenure_pipeline/` to the HPC (not only the root-level duplicate).

If a path is ignored, **plan on rsync or regeneration** on the HPC.

---

9. Per-machine configuration — *not* a separate "path file" in Git

Question: Does each machine have its own file checked into Git that tells the code where everything lives?

Answer: No. There is no committed per-host map of directories.

- **Paths inside the project** are built in `540_tenure_pipeline` **Cell 0** from `WORKSPACE_ROOT` (repo root) and `TENURE_PIPELINE_DIR` (`WORKSPACE_ROOT / "tenure" / "tenure_pipeline"`). `WORKSPACE_ROOT` is found by walking up until `functionsG_working.py` exists at the

repo root. The same code works whether the repo lives at `/Users/.../Cursor Workspace PDE` or `/home/you/pde` — only the **relative** layout under the clone must match.

- **Secrets** (API keys, email for OpenAlex polite pool) go in `.env` at the **repo root**. That file is **gitignored**; you **create or copy it on each machine** from `.env.example`. Git never stores your keys.
- Everything else (conda env name, Jupyter kernel) is **local setup**, not a special file in the repo for "machine identity."

So: **one folder tree in the clone**, **one** `.env` **per machine**, **no** duplicate Git-tracked path configuration per computer.

---

10. Same folder structure everywhere (relative paths) — what to create / rsync

You do **not** need the **same absolute path** on Mac and HPC (e.g. you do not need `/Users/charleslevine/...` on Linux). You **do** need the **same directories and files under your clone root** so that `tenure/tenure_pipeline/...` (Cell 0's `TENURE_PIPELINE_DIR`) points at real data.

Do you have to `mkdir` by hand?

Usually **no**. `rsync -a source/ dest/` creates intermediate directories on the destination. You only need to create the **parent** clone directory first (`git clone` already did that).

Canonical tree for the tenure notebook (under `WORKSPACE_ROOT`)

After `git clone`, the canonical pipeline tree is `tenure/tenure_pipeline/` (same folder as `540_tenure_pipeline.ipynb`). Cell 0 reads and writes artifacts there, for example:

Path relative to repo root	Role
`tenure/tenure_pipeline/dblp_parsed/`	DBLP-derived `dblp_*.jsonl` (Stage 1)
`tenure/tenure_pipeline/faculty_snapshots/`	Raw HTML snapshot tree (Stages 3–4)
`tenure/tenure_pipeline/faculty_snapshots_*.jsonl`, `faculty_panel*.jsonl`, `openalex_*.jsonl`, etc.	Large JSONL at pipeline root (stages 4–6)
`tenure/tenure_pipeline/r1_cs_departments.csv`, plan / index JSONL, subpage targets, IPEDS CSVs	Stage inputs/outputs as used in the notebook
`python_packages/dblp-parser/dblp.xml` (+ `dblp.dtd`)	DBLP XML if you run Stage 1 on the HPC
`datasets/tenure/`	Alternative location mentioned in `.gitignore` for DBLP XML

Practical rsync (Mac → HPC): sync the **whole** `tenure/tenure_pipeline/` directory when in doubt:

```
# Example: from your Mac, after replacing HOST and REMOTE_REPO_ROOT
rsync -avz --progress \
  "/path/to/Cursor Workspace PDE/tenure/tenure_pipeline/" \
  you@HOST:"/path/to/Cursor Workspace PDE/tenure/tenure_pipeline/"
```

Then, if you need the DBLP **XML** on the cluster:

```
rsync -avz --progress \
  "/path/to/Cursor Workspace PDE/python_packages/dblp-parser/" \
  you@HOST:"/path/to/Cursor Workspace PDE/python_packages/dblp-parser/"
```

Use the **same** `REMOTE_REPO_ROOT` path you used for `git clone` on the HPC so **relative** paths inside the project match Cell 0.

Legacy duplicate at repo root: `tenure_pipeline/`

Some checkouts still have a **second** `tenure_pipeline/` directory **next to** `tenure/` (historical duplicate). **Cell 0 no longer uses that tree** for `STAGE*` paths or imports — it uses `tenure/tenure_pipeline/`. You may delete or ignore the root-level copy after confirming nothing important lives only there; `faculty_snapshot_search_roots()` still checks the legacy `…/tenure_pipeline/faculty_snapshots` path so old HTML is discoverable during migration.

11. Path audit (hardcoded Mac paths in code?)

A quick check of `tenure/tenure_pipeline/*.py` shows **no** hardcoded `/Users/...` or `Dropbox` paths — paths are built from `Path`, repo layout, or arguments.

Notebook caveat: Saved **cell outputs** inside `540_tenure_pipeline.ipynb` may still **display** old printouts with `/Users/charleslevine/...` from earlier runs. That is **stale output text**, not the live path logic. After you open the project on the HPC and **re-run Cell 0**, new prints should show the **Linux** `WORKSPACE_ROOT`. Clearing notebook outputs before commit keeps diffs smaller (optional).

---

12. Minimal command cheat sheet

Goal	Commands
Save code to GitHub	`git add -A` → `git commit -m "..."` → `git push`
Update local clone	`git pull`
New env on HPC from committed yml	`conda env create -f environment-tenure_net.yml -n tenure_net`
Refresh big data Mac → HPC	`rsync -avz --progress source/ user@hpc:dest/`
Secrets	Create `.env` from `.env.example` on each machine

Last updated: sections 9–11 — per-machine config (`.env` vs Git), canonical `tenure_pipeline/` rsync targets, path audit. Adjust hostnames (e.g. Rivanna) to match UVA's current documentation.